PrepTech: avance de la Entrega 2

Coach de entrevistas técnicas + módulo educativo IoT. Resumen para la exposición.

Integrantes: Alvaro Sebastian Espino Jimenez, Llatas Otoya Sebastian, Lindel Junior Salvador Silva. Curso: Herramientas de Desarrollo Profesional TIC. Docente: Anthony Hans Delgado Chavarri.

1. Punto de partida (Entrega 2)

La Entrega 2 definió el diseño del Coach PrepTech: prompt de sistema, cuatro funciones (get_practice_questions, get_user_progress, save_session_feedback, get_interview_countdown), flujo de ejecución y una matriz de riesgos. El frontend (React) y el Coach (Python con Streamlit) estaban separados.

2. Qué se hizo de nuevo

Bloque	Qué se implementó
Integración	Frontend React como interfaz y backend Python como API (FastAPI, server.py). No se reescribió el Coach: la API reutiliza sus prompts, funciones y protecciones. El Coach ahora usa el progreso real del usuario, no datos de demostración.
Configuración de IA	Pantalla nueva con campo de API Key, botón Conectar y estado de conexión. La clave se valida contra OpenAI y vive solo en la memoria del backend; nunca se guarda en el navegador ni en disco y se muestra enmascarada.
Módulo IoT	Simulador propio por código (costo S/ 0) de ESP32 y Arduino Uno. Tres lecciones con verificación automática, placa animada, monitor serie, errores con número de línea y pistas con IA.
Pantallas del frontend	Coach IA, Módulo IoT, Configuración de IA y Perfil.
Calidad y despliegue	21 pruebas automáticas del backend, Docker Compose, proxy Nginx y un job de pruebas del backend en el pipeline de CI.
Correcciones	Se creó la pantalla Perfil, que no existía, y se arregló un error que bloqueaba la caja de retroalimentación.

3. Arquitectura

Frontend (React) → /api → Backend (FastAPI) → módulos: Coach (services y prompts, sin cambios) y simulador IoT (iot/). El frontend solo dibuja; toda la lógica y la clave de la API están en el backend, como pedía la Entrega 2.

4. Módulo IoT en detalle

- **Simulador:** intérprete de un subconjunto de C/Arduino (pinMode, digitalWrite/Read, analogRead, delay, Serial.println, if, for, while) con reloj virtual. Devuelve una línea de tiempo de eventos que el frontend reproduce.
- **Protecciones:** límite de instrucciones y de tiempo virtual, y aviso cuando el loop() no usa delay().
- **Lecciones:** 1) parpadear un LED, 2) botón con INPUT_PULLUP, 3) sensor analógico con monitor serie.

- **Evaluación:** cada lección se prueba con un escenario fijo (por ejemplo, botón pulsado entre 1,0 y 2,5 s) y checks automáticos.
- **Límite:** es una simulación didáctica; no modela electrónica real.

5. Presupuesto

Alternativa	Costo	Observación
Simulador propio (elegida)	S/ 0 en licencias y hardware	Integrado a la UI y a las lecciones, sin depender de terceros.
Simulador de terceros	Gratis en planes básicos; pago para uso comercial	Más realista, pero con dependencia y licencia ajenas.
Hardware real	Referencial S/ 40 a 80 por kit	Fuera de alcance: se busca aprender sin comprar nada.

Esfuerzo estimado: unas 48 horas entre tres integrantes. Los precios de terceros y de hardware son referenciales y deben confirmarse antes de la entrega. El único gasto variable es el uso de tokens de OpenAI.

6. Cambios respecto al diseño de la Entrega 2

- La Entrega 2 proponía la API de Asistentes con backend en Node. Se implementó con Chat Completions y backend en Python, conservando las mismas cuatro funciones y el mismo prompt, porque el Coach ya existía en Python y se priorizó unificar en lugar de rehacerlo.
- Todavía no hay usuarios ni inicio de sesión: el user_id es fijo y el historial no se conserva entre sesiones.

7. Guion de demostración (5 minutos)

1. Configuración de IA: pegar la clave, pulsar Conectar y mostrar el estado Conectado.
2. Coach: pedir recomendaciones y mostrar que consulta funciones con datos reales.
3. IoT, lección 1: escribir el parpadeo, compilar y ver el LED animado.
4. IoT, lección 2: provocar un error con línea, corregirlo y ver la verificación aprobada.
5. Pista con IA y cierre con el presupuesto de costo cero.

8. Pendientes y trabajo futuro

- Registro e inicio de sesión de usuarios y memoria persistente por usuario.
- Desplegar el backend junto al frontend en el servidor (Dokploy, tipo Compose).
- Más lecciones IoT (PWM, sensores de temperatura) y más preguntas en el banco.